COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE-010	**SECTION:**	CPE21S2
DATE PERFORMED:	09/15/2026	**DATE SUBMITTED:**	09/16/2026
NAME:	Julian Benedict D.R. Barizo	**INSTRUCTOR:**	Engr. Jimlord M. Quejado

ACTIVITY NO. 8.1 **[Sorting Algorithms Pt. 2]**

1. PROCEDURE OUTPUT

Code + Console Screenshots

```cpp
int main() {
    const int maxSize = 100;
    int original[maxSize];

    // generate 100 random, unsorted values
    srand(time(0));
    for (int i = 0; i < maxSize; i++) {
        original[i] = rand() % 1000;
    }

    std::cout << "Unsorted Arr" << std::endl;
    printArray(original, maxSize);
```

```cpp
// Prints all elements of the array, 10 per line
template <typename T>
void printArray(T arr[], int n) {
    for (int i = 0; i < n; i++) {
        std::cout << arr[i] << " ";
        if ((i + 1) % 10 == 0) std::cout << std::endl;
    }
    if (n % 10 != 0) std::cout << std::endl;
}
```

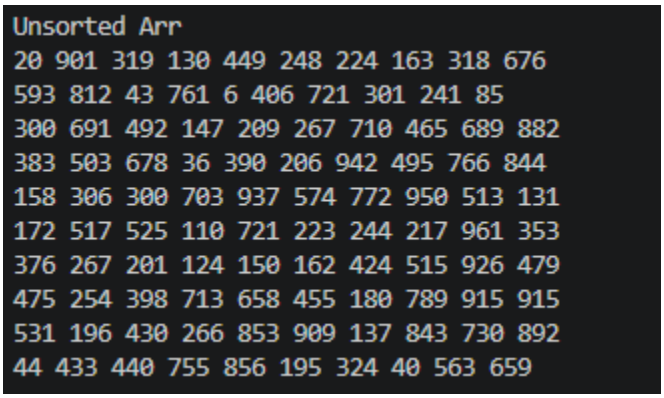
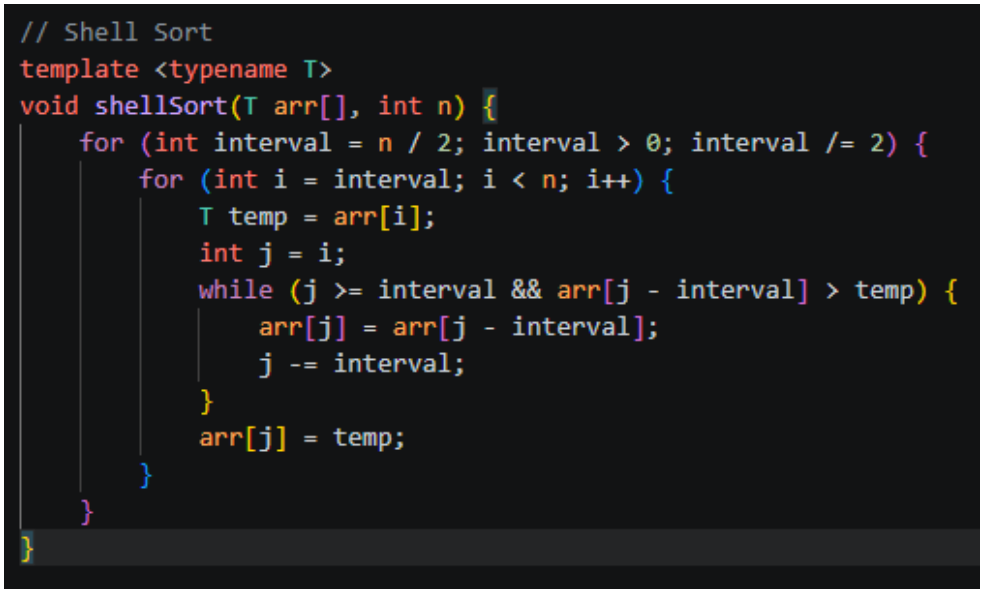
	
Observations	The best and worst case scenario for the time complexity is both **O(n)** because the function contains a singular for loop that iterates from i = 0 to i = n - 1 which executes exactly n times for any input array of size n.

Table 8-1. Array of Values for Sort Algorithm Testing

Code + Console Screenshot	

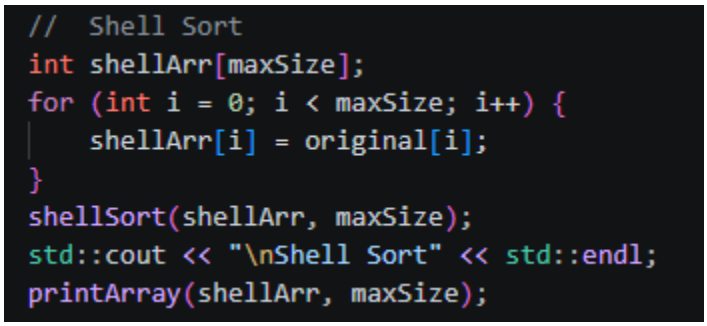
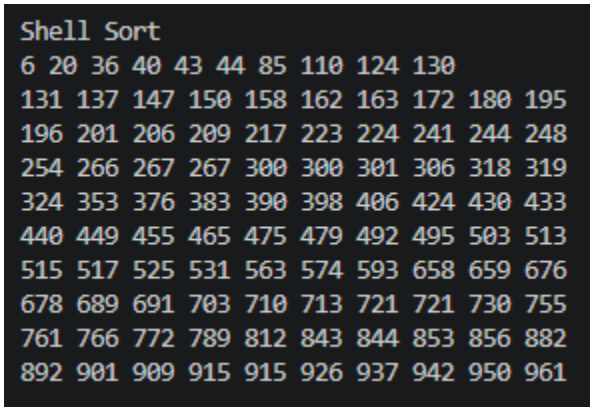
	 
Observations	This Shell Case sort function has a best case scenario of **O(n log n)** and it occurs when the input array is already sorted. However, the worst case scenario is **O(n^2)** which happens if the size of the values of the elements in your array are far apart which makes the execution of the code longer.

Table 8-2. Shell Sort Technique

Code + Console Screenshots

```cpp
// Merge Sort
template <typename T>
void merge(T arr[], int left, int middle, int right) {
    int n1 = middle - left + 1;
    int n2 = right - middle;
    std::vector<T> leftArr(n1), rightArr(n2);

    for (int i = 0; i < n1; i++) leftArr[i] = arr[left + i];
    for (int j = 0; j < n2; j++) rightArr[j] = arr[middle + 1 + j];

    int i = 0, j = 0, k = left;
    while (i < n1 && j < n2) {
        if (leftArr[i] <= rightArr[j]) arr[k++] = leftArr[i++];
        else arr[k++] = rightArr[j++];
    }
    while (i < n1) arr[k++] = leftArr[i++];
    while (j < n2) arr[k++] = rightArr[j++];
}
```

```cpp
//  Merge Sort
int mergeArr[maxSize];
for (int i = 0; i < maxSize; i++) {
    mergeArr[i] = original[i];
}
mergeSort(mergeArr, 0, maxSize - 1);
std::cout << "\nMerge Sort" << std::endl;
printArray(mergeArr, maxSize);
```

```
Merge Sort
6 20 36 40 43 44 85 110 124 130
131 137 147 150 158 162 163 172 180 195
196 201 206 209 217 223 224 241 244 248
254 266 267 267 300 300 301 306 318 319
324 353 376 383 390 398 406 424 430 433
440 449 455 465 475 479 492 495 503 513
515 517 525 531 563 574 593 658 659 676
678 689 691 703 710 713 721 721 730 755
761 766 772 789 812 843 844 853 856 882
892 901 909 915 915 926 937 942 950 961
```

Observations

Merge Sort is a sorting algorithm where you divide the array into halves until it is broken down into single element sublists, and then merging those sublists back together in the correct order .The best and worst case scenario for this Merge Sort Algorithm is both **O(n log n)** because every call cuts the array in half and calls itself on each half.

Table 8-3. Merge Sort Algorithm

Code + Console Screenshots

```cpp
// Quick Sort
template <typename T>
int partition(T arr[], int low, int high) {
    T pivot = arr[high];
    int i = low - 1;

    for (int j = low; j < high; j++) {
        if (arr[j] < pivot) {
            i++;
            std::swap(arr[i], arr[j]);
        }
    }
    std::swap(arr[i + 1], arr[high]);
    return i + 1;
}

template <typename T>
void quickSort(T arr[], int low, int high) {
    if (low < high) {
        int pivot = partition(arr, low, high);
        quickSort(arr, low, pivot - 1);
        quickSort(arr, pivot + 1, high);
    }
}

#endif
```

```cpp
    //  Quick Sort
    int quickArr[maxSize];
    for (int i = 0; i < maxSize; i++) {
        quickArr[i] = original[i];
    }
    quickSort(quickArr, 0, maxSize - 1);
    std::cout << "\nQuick Sort" << std::endl;
    printArray(quickArr, maxSize);

    return 0;
}
```

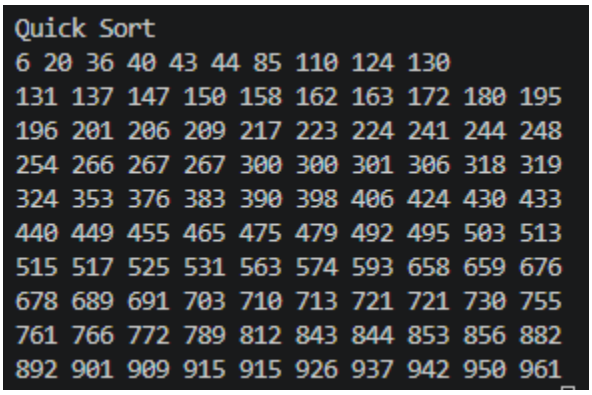
	
Observations	This Quick Sort Algorithm has the worst case time complexity of **O(n^2)** and it occurs when an array is already sorted but the pivot gets put at the very end which leaves every element on one side so instead of splitting in half, you split one element into everything else. The best case scenario of this is **O(n log n)** because if the pivot happens to land somewhere near the middle each time, you get two roughly equal halves.

Table 8-4. Quick Sort Algorithm

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Problem 1: Can we sort the left sub list and right sub list from the partition method in quick sort using other sorting algorithms? Demonstrate an example.
Yes, you can sort the left sub list and right sub list from the partition method in quick sort using other sorting algorithms. Those are two different arrays so any sorting algorithm will be able to sort them. Here is an example of the left and right sub lists getting sorted using shell sort instead of quick sort:

```cpp
#include <iostream>
#include "sortingAlgo2.h"

int main() {
    int arr[] = {4, 34, 29, 48, 53, 87, 12, 30, 44, 25, 93, 67, 43, 19, 74};
    int n = 15;

    std::cout << "Original array:\n";
    printArray(arr, n);

    //partition once, using quick sort's partition method
    int pivotIndex = partition(arr, 0, n - 1);

    std::cout << "\nAfter one partition (pivot = " << arr[pivotIndex] << ", now at index " << pivotIndex << "):\n";
    printArray(arr, n);

    //sort the left and right sub-lists using SHELL SORT instead of quick sort
    if (pivotIndex - 1 > 0)
        shellSort(arr, pivotIndex);          // left sub-list: arr[0 .. pivotIndex-1]
    if (n - (pivotIndex + 1) > 1)
        shellSort(arr + pivotIndex + 1, n - pivotIndex - 1); // right sub-list

    std::cout << "\nFinal array after sorting sub-lists with Shell Sort:\n";
    printArray(arr, n);

    return 0;
}
```

```
Original array:
4 34 29 48 53 87 12 30 44 25
93 67 43 19 74

After one partition (pivot = 74, now at index 12):
4 34 29 48 53 12 30 44 25 67
43 19 74 87 93

Final array after sorting sub-lists with Shell Sort:
4 12 19 25 29 30 34 43 44 48
53 67 74 87 93
```

This confirms that the sub lists produced by the partition can be sorted by a different algorithm which sorts each side independently and still results to the full array being sorted properly.

Problem 2: Suppose we have an array which consists of {4, 34, 29, 48, 53, 87, 12, 30, 44, 25, 93, 67, 43, 19, 74}. What sorting algorithm will give you the fastest time performance? Why can merge sort and quick sort have O(N • log N) for their time complexity?

For an array with only 15 elements it barely matters, but in practice an algorithm like Shell Sort would most likely win because it has better theoretical complexity. They both O(n log n) because each cuts the array roughly in half every recursive step. And those steps runs for log N levels, and each level does O(n) total work meaning if you multiply them you get O(n log n) time complexities.

Conclusion
In conclusion, this activity helped me better my understanding of these three extra shell sorts which are Merge Sort, Shell sort, and Quick sort and how they can differ from each other and how they can be implemented in different scenarios. It also made me know about the different advantages and disadvantages of each algorithm making me more knowledgeable in knowing what algorithm to use depending on each scenario to be able to optimize my code. They all output the same sorted elements in the end, but the difference is the time it takes for them to sort the elements in each iteration.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **Claude**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **Code Debugging, Code Guidance**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **Told me how to start my code, and told me the errors I was making**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.